

COMPUTER SCIENCE TRIPOS Part IB – 2024 – Paper 4

8 Cybersecurity (fms27)

Consider the following hacking game. You are given access, as user `player`, to a 32-bit x86 machine. The C program `challenge` (setuid `boss`) accepts user input and contains a buffer overflow vulnerability. The stack is executable, there are no stack canaries in the binary and ASLR is turned off. When the `challenge` program is running, the first byte of its vulnerable buffer (always word-aligned) may appear in memory anywhere within a 32768 byte region starting at `0xffff6000`. The vulnerable buffer is 100 bytes long and the C function that contains it has only two more local variables, both `char`. The `challenge` program truncates its input at 2048 bytes.

Your input to the `challenge` program, including a specific 50-byte payload, is built by the following Python function. You supply five parameters to `buildInput` to create an input and then the `challenge` program is run on the resulting input, but doing so costs you \$0.01 per byte of generated input plus \$50 per run. You may repeat this as needed. You win a large prize if you cause the payload to be executed under `userid boss`. You seek a winning strategy that maximises your profit.

```
def buildInput(length, payloadOffset, retOffset, retAddress, retCopies):
    payload = bytes(b"\x31...") # 50 bytes of evil x86 machine code
    input = bytearray([0x90] * length) # prefill with nop
    input[payloadOffset:payloadOffset+len(payload)] = payload
    for j in range(retCopies): # stack spraying
        input[retOffset+(j*4):retOffset+(j*4+1)] = \
            retAddress.to_bytes(4, byteorder='little')
    return input
```

- (a) Explain whether you would favour more runs or longer inputs. [1 mark]

Answer: With many small inputs we pay the \$50 per-run fee more times. There isn't a trade-off whereby smaller inputs make us save on another front—on the contrary, there is a non-proportionality due to the fact that we need to “waste” the first 100 bytes of input to even *get* to the place where we can overwrite the return address on the stack. So there is no reason to like short inputs and we must instead prefer to use as few inputs as possible, each as large as possible (2048 bytes), to cover all the places where the return address might be.

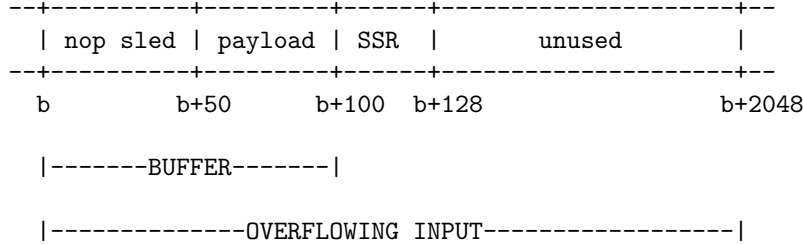
- (b) Explain whether, in your input, you would put the nop sled and payload before or after the stack spraying region. [3 marks]

Answer: Since the stack spraying region is in a fixed position, about 100 bytes into our buffer, to maximise the length of our nop sled we want to place the sled *after* the stack spraying region, where it can be almost 1900 bytes, instead of before, where it cannot be longer than 100 bytes (in fact just 50 if the payload immediately follows it).

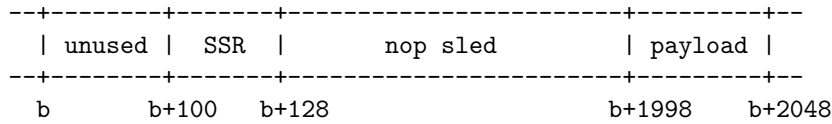
— *Solution notes* —

[*Note:* Here and elsewhere, diagrams welcome but not required for full marks, provided the answer is sufficiently clear. Memory regions not drawn to scale. SSR = stack spraying region.]

Nop sled before stack spraying region: 50 bytes (buffer size, 100, minus payload size, 50).

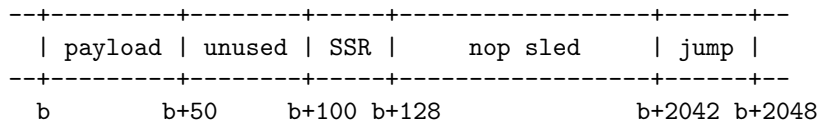


Nop sled after stack spraying region: $1998-128 = 1870$ bytes.



The following is not required for full marks. The position of the nop sled relative to the stack spraying region is important, but whether the payload itself immediately follows the nop sled or not makes little difference in practice. Why? If we stick the payload at the very start of the buffer, to avoid wasting otherwise unused space, and put, at the very end of our input, a relative jump back to the start of the buffer (where our payload now is), we can extend the nop sled by almost 50 bytes (50 minus the length of the relative jump instruction, which is 2 bytes for the opcode and 4 bytes for the displacement—pedantic details not relevant and not expected from the students). However, the 44 bytes per run we save this way are too few to make a difference, so this micro-optimisation is not worth bothering with—we’ll still need the same number of runs as we’ll derive in Part (d), and we’ll save less than 1% of the total budget required to guarantee a win.

Longest possible nop sled: $2042-128=1914$ bytes.

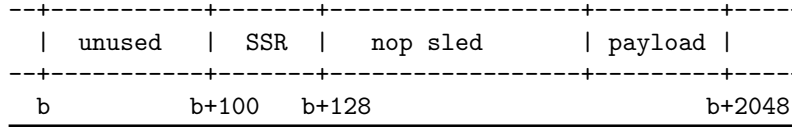


- (c) Explain whether you would favour a longer nop sled or a longer stack spraying region. [3 marks]

Answer: Since ASLR is disabled, *ceteris paribus* the buffer will appear at the same memory address at every run: it may be anywhere in that 32 kiB region, but it’s always in the same place. We just don’t know where. What we change from one run to the next is our *guess* of where that place is, because the absolute address of our payload (which is, to a first approximation, the numeric value we need to poke into the return address) depends on it.

We don’t know the *absolute* address of the word on the stack with the return address, but we have a somewhat better idea of where it is *relative* to the start of the buffer: namely, slightly more than 100 bytes away, since the other two local variables, whether they appear before or after, don’t take up much space. For good measure, and for round numbers, we can safely assume that the return address is located in memory between $b + 100$ and $b + 128$ bytes, if b is the absolute address of the first byte of the buffer. So our stack spraying region can be small, and simply cover those 7 words ($\frac{128-100}{4} = 7$), and we can use the rest of our input for

a longer nop sled, to absorb our greater uncertainty about the absolute address of the buffer (and therefore of our evil payload).



- (d) Outline, with justification, an efficient strategy for guaranteeing a win in this game, detailing among other things how many *runs* (invocations of the **challenge** program) it involves in the worst case. [5 marks]

Answer: To exploit the buffer overflow vulnerability we must overwrite the return address on the stack with the address of our payload, or at least with some address within a nop sled leading to our payload.

As we explain in Part (c), in absence of ASLR the buffer always loads at the same absolute address in the various runs. We don't know that address but, at every run, we can use one that covers as many possible addresses as there are bytes in our sled (plus one, to be pedantic; but let's not be). Thus, if the buffer may start anywhere within a 32 kiB region, so can our payload, but at every new run we may guess (assume) a new starting address for our nop sled and consequently a new return address to spray in the relevant positions on the stack. The buffer does not move between runs, but our guess changes. One of these times it will be correct, and that time the **ret** instruction at the end of the C function with the vulnerable buffer will finally send the flow of control through our nop sled and onto our payload.

In the worst case, the maximum number of runs we need to cover all possible guesses is thus 32 kiB divided by the length of our nop sled, which we therefore want to make as long as possible. With the reasoning from Part (c), our nop sled starts at byte 128 and ends before byte 2048 - 50 (leaving space at the end for the payload), which makes it exactly 1870 bytes long. It will therefore take $\lceil 32768/1870 \rceil = 18$ runs to guarantee victory.

- (e) Give, with justification, the specific values of each of the five input parameters your strategy will pass to **buildInput()** during run *i*, with runs numbered from 0 onwards. The values may be constant or parametric in *i*. [8 marks]

Answer:

`length = 2048,`

the maximum possible, for the reasons given in Part (a).

`payloadOffset = length - payloadLength = 2048 - 50 = 1998,`

placing the payload at the very end of the input, for the reasons given in Part (b).

`retOffset = 100,`

for the reasons given in Part (c).

`nopSledOffset = 128,`

for the reasons given in Part (d).

`nopSledLength = payloadOffset - nopSledOffset = 1998 - 128 = 1870,`

for the reasons given in Part (d).

`retAddress = 0xffff6000 + payloadOffset + i * nopSledLength,`

for the reasons given in Part (d).

`retCopies = 7`,
for the reasons given in Part (c).

We start by guessing that the buffer is at the lowest possible address (0xffff6000). If it were, then our payload would start `payloadOffset = 1998` bytes after that (50 bytes from the end of our input, which is at offset 2048). We overwrite the return address with a `retAddress` that points at where the payload starts (initially, in run 0, `0xffff6000+payloadOffset`). If our guess about the position of the buffer was correct, the payload executes. If our guess was slightly wrong and the buffer is slightly higher, then the address we guessed ends up being slightly lower than the start of our payload. It therefore falls into our nop sled, and our payload still executes. This works for every address in the nop sled. If none of them worked, meaning our guess was too far below the actual absolute address of the buffer in memory (by more than the length of our sled), so it's time to pay \$50 for another run with another guess, trying a return address that will be good for the next batch of actual starting points for the buffer.
